

CS 613200 Advanced Logic Synthesis Homework 1 (2026, Fall)

賈俊佑 studentID:114062645

1. Introduction

這個作業的目標是評估與分析兩套 synthesis tool，SIS 與 ABC 的最佳化能力。比較這兩種 tool 的主要指標包括執行時間，以及最佳化後電路的 node 數量與 level 數量。在實驗過程中，將分別使用這兩套工具中不同的優化方式，並透過相同的評估指標進行比較，以分析各種方法在最佳化效果上的差異與表現。

2. Experimental Setup

2.1 ABC 實驗設置

在 ABC 的實驗中，主要包含一個 baseline 與五種不同的最佳化方式，分別如下：

1. Traditional AIG Flow

```
read $filepath; strash; rewrite; refactor; balance; rewrite; print_stats
```

透過 rewrite、refactor 與 balance 等操作，對電路結構進行重組與化簡，以降低節點數與邏輯深度。

2. Orchestrate

```
read $filepath; strash; orchestrate; print_stats
```

Greedy algorithm，為一種自動化的最佳化方法。

3. Pure syn4

```
read $filepath; strash; &get; &syn4; &ps
```

ABC9 的 GIA-based flow，先透過 &get 進入 GIA 表示，再使用 &syn4 進行最佳化，最後以 &ps 輸出統計資訊。

4. dch + syn4

```
read $filepath; strash; &get; &dch; &syn4; &ps
```

與 Pure syn4 相比，這個方法在 &syn4 之前先加入 &dch，也就是先進行額外的化簡和處理，再進一步做 syn4 最佳化。

5. DeepSyn

```
read $filepath; strash; &get; &deepsyn -J 300 -T 300; &ps
```

同樣屬於 ABC9 flow。其中 -J 300 表示當連續 300 次迭代沒有改善時停止，-T 300 表示整體執行時間上限為 300 秒。此方法為自動化高強度最佳化操作流程，可以在較高的計算成本下，進行更深入的搜尋與

最佳化。

2.2 SIS 實驗設置

在 SIS 的實驗中，主要包含一個 **baseline** 與三種不同的最佳化方式，實驗中所使用的 **script.algebraic**、**script.boolean** 與 **script.rugged**，皆為 SIS 內建提供的 **script**，可在 SIS 的 **script library** 中找到，分別如下：

1. Algebraic Flow (script.algebraic)

```
read_blif $filepath; source $SIS_LIB/script.algebraic
```

此方法主要偏向利用代數式重組與邏輯抽取的方式來減少電路中的冗餘結構，進而降低 **node** 數量。

2. Boolean Flow (script.boolean)

```
read_blif $filepath; source $SIS_LIB/script.boolean
```

此方法主要著重於從 **Boolean function** 本身做更進一步的化簡，以降低電路中的冗餘邏輯。

3. Rugged Flow (script.rugged)

```
read_blif $filepath; source $SIS_LIB/script.rugged
```

使用一種較通用且較強的內建 **script**，會綜合多種化簡方式來進一步壓縮電路結構。

3. Experimental Results

為了方便後續對 SIS 與 ABC 的結果進行一致性的比較，統一以 **AIG node** 數量與 **logic level** 作為主要比較指標。此外，為避免個別 **benchmark** 在最佳化流程中耗費過長的執行時間，若單一方法超過 30 分鐘(有些跑好幾個小時也跑不出來)，則視為 **timeout**，並停止該方法的執行與記錄。

3.1 Node 數量

Benchmark	Category	Base	ABC_Trad	ABC_Orch	ABC_PS4	ABC_Dch+PS4	ABC_DeepSyn	SIS_Alg	SIS_Bool	SIS_Rug
isqrt16.aig	arithmetic	4598	3741	3803	3574	3696	2020	4234	TIMEOUT	TIMEOUT
isqrt8.aig	arithmetic	106	59	66	59	49	35	91	72	66
smul8.aig	arithmetic	88987	70796	71707	72678	74894	61477	TIMEOUT	TIMEOUT	TIMEOUT
square12.aig	arithmetic	11077	9146	9207	9465	9541	6599	10644	TIMEOUT	TIMEOUT
udiv5.aig	arithmetic	421	325	330	286	298	123	348	382	325
udiv8.aig	arithmetic	6217	4952	5059	4935	5018	2706	5694	6300	TIMEOUT
umul5.aig	arithmetic	1872	1464	1450	1546	1529	760	1713	1873	TIMEOUT
umul7.aig	arithmetic	30474	24428	24693	25348	25725	18233	28722	TIMEOUT	TIMEOUT
umul8.aig	arithmetic	120875	96882	98207	99145	102178	86380	TIMEOUT	TIMEOUT	N/A
usquare14.aig	arithmetic	43122	35553	35975	36576	37856	28612	41302	TIMEOUT	TIMEOUT
arbiter.aig	random_control	11839	11839	11839	4799	5060	2298	6275	4944	4738
cavlc.aig	random_control	693	681	674	670	661	419	720	776	751
ctrl.aig	random_control	174	119	103	94	100	70	126	101	102
dec.aig	random_control	304	304	304	304	304	304	304	304	304
i2c.aig	random_control	1342	1227	1200	1051	1027	693	1060	1017	968
int2float.aig	random_control	260	219	216	214	207	141	223	214	198
mem_ctrl.aig	random_control	46836	46304	45809	39711	37250	9296	49572	48513	N/A
priority.aig	random_control	978	729	771	820	675	385	990	888	583
router.aig	random_control	257	236	245	190	168	113	260	210	194
voter.aig	random_control	13758	10513	9516	14388	15295	7931	12349	10917	11013
Average improvement (%)	-	0.00%	16.95%	16.92%	21.03%	21.82%	49.83%	9.90%	15.33%	26.37%

3.2 Level 數量

Benchmark	Category	Base	ABC_Trad	ABC_Orch	ABC_PS4	ABC_Dch+PS4	ABC_DeepSyn	SIS_Alg	SIS_BooI	SIS_Rug
isqrt16.aig	arithmetic	32	24	30	20	19	18	32	TIMEOUT	TIMEOUT
isqrt8.aig	arithmetic	14	9	13	8	8	7	13	24	23
smul8.aig	arithmetic	65	32	39	25	24	23	TIMEOUT	TIMEOUT	TIMEOUT
square12.aig	arithmetic	38	23	32	20	18	16	35	TIMEOUT	TIMEOUT
udiv5.aig	arithmetic	20	16	18	13	13	14	20	21	22
udiv8.aig	arithmetic	32	28	32	24	22	20	31	38	TIMEOUT
umul5.aig	arithmetic	23	18	23	15	16	16	25	29	TIMEOUT
umul7.aig	arithmetic	46	28	37	22	22	20	47	TIMEOUT	TIMEOUT
umul8.aig	arithmetic	69	33	43	26	24	24	TIMEOUT	TIMEOUT	N/A
usquare14.aig	arithmetic	67	28	45	22	21	20	60	TIMEOUT	TIMEOUT
arbiter.aig	random_control	87	87	87	24	48	29	35	64	64
cavlc.aig	random_control	16	16	16	15	13	18	19	25	19
ctrl.aig	random_control	10	9	10	5	6	9	10	20	22
dec.aig	random_control	3	3	3	3	3	3	3	3	3
i2c.aig	random_control	20	16	18	12	13	17	18	16	15
int2float.aig	random_control	16	15	16	13	10	11	16	18	19
mem_ctrl.aig	random_control	114	114	114	76	77	52	125	132	N/A
priority.aig	random_control	250	223	226	124	101	67	266	186	153
router.aig	random_control	54	26	42	20	19	14	38	34	35
voter.aig	random_control	70	58	63	64	61	65	76	110	109
Average improvement (%)	-	0.00%	23.54%	11.05%	40.42%	41.39%	41.27%	4.10%	-18.13%	-14.73%

3.3 Runtime(s)

Benchmark	Category	Base	ABC_Trad	ABC_Orch	ABC_PS4	ABC_Dch+PS4	ABC_DeepSyn	SIS_Alg	SIS_BooI	SIS_Rug
isqrt16.aig	arithmetic	0.05	0.13	0.14	0.13	0.52	300.15	3.49	TIMEOUT	TIMEOUT
isqrt8.aig	arithmetic	0.04	0.07	0.07	0.07	0.07	4.62	0.02	0.02	0.02
smul8.aig	arithmetic	0.09	1.44	2.34	1.54	16.51	301.19	TIMEOUT	TIMEOUT	TIMEOUT
square12.aig	arithmetic	0.05	0.22	0.76	0.37	1.3	303.42	16.35	TIMEOUT	TIMEOUT
udiv5.aig	arithmetic	0.04	0.07	0.07	0.07	0.09	26.93	0.12	0.13	0.38
udiv8.aig	arithmetic	0.04	0.14	0.16	0.15	0.78	302	5.8	305.97	TIMEOUT
umul5.aig	arithmetic	0.05	0.09	0.11	0.09	0.2	218.11	0.85	2.92	TIMEOUT
umul7.aig	arithmetic	0.06	0.49	0.64	0.56	3.73	305.45	143.86	TIMEOUT	TIMEOUT
umul8.aig	arithmetic	0.12	2.08	3.51	2.48	22.15	334.4	TIMEOUT	TIMEOUT	N/A
usquare14.aig	arithmetic	0.07	0.7	1	0.76	6.36	312.2	360.36	TIMEOUT	TIMEOUT
arbiter.aig	random_control	0.06	0.24	0.25	0.11	3.52	300.71	18.01	49.18	39.14
cavlc.aig	random_control	0.04	0.08	0.08	0.08	0.12	123.14	0.3	0.82	1.18
ctrl.aig	random_control	0.1	0.07	0.07	0.07	0.08	7.54	0.03	0.04	0.05
dec.aig	random_control	0.04	0.08	0.08	0.04	0.06	16.2	0.05	0.09	0.11
i2c.aig	random_control	0.04	0.08	0.09	0.08	0.15	91.88	0.32	0.4	0.43
int2float.aig	random_control	0.04	0.07	0.07	0.07	0.08	23.54	0.04	0.06	0.16
mem_ctrl.aig	random_control	0.07	0.63	0.88	0.55	9.05	300.48	871.19	1083.68	N/A
priority.aig	random_control	0.04	0.09	0.08	0.05	0.11	59.51	0.22	0.15	1.09
router.aig	random_control	0.05	0.08	0.07	0.07	0.09	6.47	0.04	0.04	16.56
voter.aig	random_control	0.05	0.46	1.1	0.33	2.88	301.43	106.14	67.74	8.9
Average increase vs ABC_Trad (%)	-		0.00%	33.31%	-3.89%	414.26%	83000.83%	14696.50%	30702.59%	3825.06%

Runtime 表最後一列以 ABC_Trad 作為基準，其中正值表示執行時間增加，負值表示執行時間減少。

註: N/A 代表跑的時候發生了 segmentation fault ,TIME OUT 則是跑了超過 30 分鐘強制被停止

4. Discussion

根據實驗結果可以看出，若統一以 AIG node 數量與 logic level 作為比較標準，ABC 的整體表現優於 SIS。這也反映出兩套工具的最佳化方向不同。ABC 較偏向針對 AIG 與 GIA 結構進行最佳化，因此在本研究的評估方式下更有優勢；相對地，SIS 主要針對 BLIF 與 multi-level network 進行最佳化，雖然在自身的表示方式下可以有效壓縮電路，但轉成 AIG 之後，這些優勢不一定能完整保留。因此，SIS 在原始網路上的化簡效果，未必會直接反映在最後用來比較的 AIG 指標上。

若觀察 ABC 各種方法的差異，可以發現它們在最佳化品質與執行時間之間有明顯的取舍。**Traditional AIG Flow** 的改善幅度雖然不是最大，但速度快且表現穩定，因此可以視為一種效率高、適合作為基準的最佳化方式。

Orchestrate 也是自動化方法，但從結果來看，其改善幅度沒有特別突出，表示這種策略雖然能自動完成最佳化流程，但不一定能在所有 **benchmark** 上帶來明顯優勢。**Pure syn4** 在 **node**、**level** 與 **runtime** 之間取得較好的平衡，顯示這個方法不只改善效果不錯，執行成本也相對合理。**dch + syn4** 能再進一步改善結果，但執行時間也明顯增加，代表額外的化簡步驟確實有幫助，只是代價更高。**DeepSyn** 則屬於高成本但高效果的方法，雖然在部分 **benchmark** 上有最好的結果，但所需時間遠高於其他方法，因此更適合用在特別重視最佳化品質的情況，而不是一般的快速比較流程。

在 SIS 的三種內建 **script** 中，**script.algebraic**、**script.boolean** 與 **script.rugged** 也呈現出不同特性。從實驗結果來看，**script.algebraic** 的表現相對穩定，較符合其以代數式重組為主的特性；**script.boolean** 著重於 **Boolean function** 的化簡，部分 **benchmark** 上雖然能帶來改善，但整體表現不夠穩定；**script.rugged** 則屬於較強、較綜合型的內建最佳化流程，但在實驗中也出現了 **TIMEOUT** 與 **N/A** 的情況，代表有些 **benchmark** 執行時間過長，或執行時發生 **segmentation fault**。這也說明 SIS 的部分方法雖然理論上能做更強的化簡，但在較大或較複雜的 **benchmark** 上，穩定性和效率仍然是限制。

從這次實驗也可以更清楚看出，不同方法的用途其實很明確。若希望在較短時間內取得穩定且不錯的結果，**Traditional AIG Flow** 與 **Pure syn4** 會是較合理的選擇；若更重視最佳化品質，則可以考慮 **dch + syn4** 或 **DeepSyn**，但必須接受更高的時間成本。換句話說，這次比較不只是看哪一個方法最好，而是看不同方法分別適合什麼樣的需求。

5. Conclusion

整體來看，這次實驗顯示不同最佳化方法在效果與執行時間之間都有明顯差異。在 ABC 的幾種 **flow** 中，**Traditional AIG Flow** 速度快且表現穩定，**Pure syn4** 在效果與效率之間取得較好的平衡，而 **dch + syn4** 與 **DeepSyn** 雖然能進一步改善結果，但也需要付出更高的時間成本。

相較之下，SIS 的三種內建 **script** 雖然也能對電路進行最佳化，但整體效果與穩定性仍不如 ABC。若以 **AIG node** 與 **logic level** 作為主要比較標準，ABC 在這次實驗中的表現整體較好；而 SIS 則較適合用來觀察不同邏輯化簡方式的特性與差異。